4. Tools and Technologies Used

Overview

This project utilizes various tools and technologies to perform end-to-end API testing of the Fake Store REST API. These tools help in creating API requests, validating responses, automating test execution, generating reports, and managing test environments efficiently.

The combination of Postman, Newman CLI, Node.js, JavaScript, and Fake Store API provides a complete API testing ecosystem that closely resembles real-world software testing practices used in the IT industry.

4.1 Postman

Description

Postman is a popular API testing tool used by developers and testers to create, execute, validate, and automate API requests. It provides a user-friendly interface for testing REST APIs without writing extensive code.

In this project, Postman serves as the primary tool for API testing.

Purpose of Using Postman

- Create API requests.
- Send HTTP requests to the server.
- Validate API responses.
- Organize APIs into collections.
- Manage environment variables.
- Write automated test scripts.
- Execute multiple requests using Collection Runner.

Features Used in This Project

Request Creation

Created requests for:

- GET Requests
- POST Requests

- PUT Requests
- PATCH Requests
- DELETE Requests

Collection Management

Organized APIs into folders:

- Authentication
- Products
- Categories
- Cart
- Users

This improves project structure and maintainability.

Environment Variables

Used environment variables to store reusable data such as:

- Base URL
- Product ID
- User ID
- Cart ID
- Authentication Token

Benefits:

- Easy maintenance
- Faster execution
- Reduced hardcoding

Test Scripts

Implemented JavaScript assertions in the Tests section to validate:

- Status Codes
- Response Time
- Response Body
- JSON Structure
- Environment Variables

Example:

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});
```

Collection Runner

Used Collection Runner to:

- Execute all requests together
- Validate complete API workflow
- Analyze execution results
- Identify failed test cases

Benefits of Postman

- Easy to use
- Supports automation
- User-friendly interface
- Supports environment management
- Provides detailed execution reports

4.2 Newman CLI

Description

Newman is the command-line collection runner for Postman. It allows Postman collections to be executed outside the Postman application through the command line.

Newman is widely used in Continuous Integration and Continuous Deployment (CI/CD) environments for automated API testing.

Purpose of Using Newman

- Execute Postman collections from Command Prompt.
- Automate API test execution.
- Generate execution reports.
- Support CI/CD pipelines.
- Analyze test results.

Newman Commands Used

Execute Collection

```
newman run FakeStore_API_Collection.json
```

Execute Collection with Environment

```
newman run FakeStore_API_Collection.json -e FakeStore_Environment.json
```

Generate HTML Report

```
newman run FakeStore_API_Collection.json -e FakeStore_Environment.json -r
cli,htmlextra
```

Benefits of Newman

- Supports automated execution.
- Easy integration with Jenkins and CI/CD tools.
- Generates detailed reports.
- Faster collection execution.
- Provides execution summary.

4.3 Node.js

Description

Node.js is a JavaScript runtime environment used to install and execute Newman.

Since Newman is distributed as an NPM package, Node.js must be installed before using Newman.

Purpose of Using Node.js

- Install Newman package.
- Install Newman HTML Reporter.
- Execute Newman commands.

Commands Used

Verify Node.js Installation

```
node -v
```

Verify NPM Installation

```
npm -v
```

Install Newman

```
npm install -g newman
```

Install HTML Reporter

```
npm install -g newman-reporter-htmlextra
```

Benefits of Node.js

- Lightweight runtime environment.
- Supports package management through NPM.
- Enables Newman execution.
- Widely used in automation projects.

4.4 JavaScript

Description

JavaScript is used within Postman test scripts to perform automated validations on API responses.

It allows testers to write assertions and automate response verification.

Purpose of Using JavaScript

- Validate status codes.
- Validate response data.
- Validate response time.
- Extract values from responses.
- Store environment variables.

Example Script

```
pm.test("Response time is less than 2000ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});
```

Benefits of JavaScript

- Supports automated validation.
- Reduces manual testing effort.
- Improves testing accuracy.
- Enables dynamic test execution.

4.5 Fake Store REST API

Description

Fake Store API is a free RESTful API that simulates a real-world e-commerce application. It provides endpoints for products, categories, carts, users, and authentication.

This API serves as the application under test (AUT) for this project.

Official Website

https://fakestoreapi.com

Modules Used

Authentication

- Login User

Products

- Get All Products
- Get Single Product
- Add New Product
- Update Product
- Delete Product

Categories

- Get All Categories
- Get Products by Category

Cart

- Get All Carts
- Get Single Cart
- Add Cart
- Update Cart
- Delete Cart

Users

- Get All Users
- Get Single User

Benefits of Fake Store API

- Free to use.
- No registration required.
- Supports CRUD operations.
- Real-world e-commerce data.
- Suitable for API testing practice.

Technology Stack Summary

Tool/Technology	Purpose
Postman	API Request Creation, Validation, Automation
Newman CLI	Command-Line Collection Execution
Node.js	Runtime Environment for Newman
JavaScript	Writing Automated Test Scripts
Fake Store API	Application Under Test (AUT)
JSON	Request and Response Data Format
REST API	Communication Architecture

Summary

This project uses Postman as the primary API testing tool, Newman CLI for automated execution and report generation, Node.js for package management and runtime support, JavaScript for automated validations, and Fake Store API as the application under test. Together, these technologies provide a complete API testing solution and demonstrate practical testing skills commonly required in software testing and QA roles.